

2. Übung – Orbitpropagation mit SGP4 (20 Punkte)

Das SGP4 Bahnmodell wird verwendet, um den Orbit von Satelliten vorherzusagen. Ausgangspunkt der Berechnung sind die Orbit Mean-Elements Message, mit denen Sie bereits in Übung 1 gearbeitet haben. Eine Beschreibung des SGP4 Modells finden Sie im SPACETRACK REPORT NO. 3 (<https://celestrak.com/NORAD/documentation/spacetrk.pdf>). Als Ergebnis liefert SGP4 die Position und Geschwindigkeit eines Satelliten im ECI System.

Benutzen Sie zur Bearbeitung dieser Übung die auf wueCampus zur Verfügung gestellte C++ Implementierung des SGP4-Modells.

Aufgabe 2.1.

4 Punkte

Machen Sie sich mit dem Code vertraut und verifizieren Sie die Implementierung mit dem im SPACETRACK REPORT NO. 3 angegebenen Test Case. Die eigenen Ergebnisse sollen auf wenigstens 1 km bzw. $0,1 \text{ km s}^{-1}$ mit dem vorgegebenen Test Case übereinstimmen. Anschließend berechnen Sie Position und Geschwindigkeit von SONATE-2 für 0, 360, 720, 1080 und 1440 Minuten nach der Epoche des OMMs. Das OMM von SONATE-2 und dem Testcase sind in `omm.txt` gegeben.

Abgabe: Die modifizierte `main.cpp`, ihre `omm.txt`, sowie je eine Textdatei mit den Ergebnissen für den Test Case und SONATE-2.

Aufgabe 2.2.

16 Punkte

Sie haben in der Vorlesung gelernt, wie man den Subsatellitenpunkt und die Höhe eines Satelliten berechnet. Entscheidend ist dabei, ob die Erde als Kugel oder als Ellipsoid betrachtet wird.

- a) Schreiben Sie in C++ ein Programm, das den Subsatellitenpunkt und die Höhe, sowohl für eine kugelförmige Erde als auch für einen Ellipsoiden berechnet. Die ECI Position des Satelliten erhalten Sie mit dem Programm aus Aufgabe 2.1. für ein beliebiges OMM. Erweitern Sie dieses Programm mit mindestens folgenden Funktionen:

- `double computeJD(int year, double dayFractions)`: Diese Funktion berechnet das Julianische Datum mit Hilfe des Jahres- und Tagesbruchteils aus dem OMM. Achten Sie bei der Berechnung auf eine ausreichende Genauigkeit! Geben Sie die Quelle ihrer verwendeten Berechnungsformel im Kommentar der Funktion mit an.
- `double computeGMST(double jd)`: Diese Funktion berechnet die Sternzeit des Greenwich Meridians θ_g (engl. Greenwich Mean Sideral Time (GMST)) zum gegebenen Julianischen Datum in radian. Reduzieren Sie das Ergebnis auf das Intervall $[0, 2\pi)$.

- `GeocentricCoordinate convertECIToGeocentric(const ECICoordinate &eciCoord, double jd)`: Wandelt eine Koordinate im ECI System zum Zeitpunkt des Julianischen Datums in ihre entsprechende geozentrische Koordinate um.
- `GeodeticCoordinate convertECIToGeodetic(const ECICoordinate &eciCoord, double jd)`: Wandelt eine Koordinate im ECI System zum Zeitpunkt des Julianischen Datums in ihre entsprechende geographische (geodätische) Koordinate um. Als Referenzellipsoid soll hierfür der WGS84 verwendet werden.

Die geozentrischen und geographische Koordinaten sollen dabei mindestens die folgenden Variablen mit den angegebenen Einheiten und Definitionsbereichen besitzen:

- `latitude`: Breitengrad in radian, im Intervall $[-\pi/2; \pi/2]$
- `longitude`: Längengrad in radian, im Intervall $(-\pi; \pi]$
- `height`: Höhe in Kilometern

Die o.g. Funktionsköpfe sind in `timeDate.h` und `coordinates.h` gegeben; Structs müssen ergänzt werden. Implementieren Sie die Funktionen in entsprechenden `.cpp` Dateien.

Bitte beachten Sie, dass der verwendete Algorithmus zur Berechnung der Werte klar in der Abgabe zu erkennen sein muss, gegebenenfalls mit einer separaten Herleitung. Denken Sie an sinnvolle Kommentare in Ihrem Code.

- b) Berechnen Sie für einen Zeitraum von 100 Minuten ausgehend der Epoche des OMMs den Subsatellitenpunkt und die Höhe, sowohl für eine kugelförmige Erde als auch für einen Ellipsoiden für das angegebene OMM von SONATE-2.

Speichern Sie die Ergebnisse in eine `.csv` Datei und geben Sie diese ab. Enthalten sein sollen in 5 Minuten Zeitabständen die folgenden Werte:

- Vergangene Zeit seit der Epoche des OMMs [min]
- ECI Position des Satelliten [km]
- Julianisches Datum mit mindestens 8 Nachkommastellen [days]
- Greenwich Mean Sidereal Time [deg]
- Breitengrad [deg], Längengrad [deg] und Höhe [km] für kugelförmige Erde
- Breitengrad [deg], Längengrad [deg] und Höhe [km] für Ellipsoid

- c) Vergleichen Sie die geozentrische und geographischen Koordinaten indem Sie den absoluten Fehler zwischen den einzelnen Komponenten (Breitengrad, Längengrad und Höhe) für das Zeitintervall der vorherigen Teilaufgabe plotten. Überlegen Sie sich sinnvolle Zeitabstände (evtl. kürzer als 5 min). Begründen Sie den groben Verlauf des Fehlers. Achten Sie auf eine aussagekräftige Skalierung der Achsen. Gegebenenfalls fügen Sie Plots mit Detailansichten hinzu.



- d) Zentrales Element vieler Programme zur Orbitvisualisierung ist die Anzeige von Bodenspuren. Erstellen Sie eine statische 2D-Darstellung der Bodenspur von SONATE-2 für die ersten drei Orbits nach der Epoche des OMMs. Nutzen Sie die geographischen Koordinaten. Wählen Sie eine geeignete Projektion und begründen Sie ihre Wahl. Denken Sie auch an passende Skalen und Hilfslinien.

Abgabe: Code (`main.cpp`, `timeDate.cpp`, `coordinates.h`, `coordinates.cpp`), Werte als `.csv`, alle Plots und Darstellungen mit entsprechenden Erläuterungen.

Hinweise zur Abgabe

- Abgabefrist: Mittwoch, 3. Juni 2026 10:00 Uhr online über WueCampus als zip Archiv
- Dateinamenkonvention: `Nachname1_Nachname2_Nachname3_Nachname4.zip`
Schreiben Sie alle Mitglieder Ihrer Gruppe auch als Kommentare in den Programmcode.
- Arbeiten Sie in Gruppen von 3 oder 4 Studierenden.
- Fügen Sie dem Archiv für jede Aufgabe die jeweiligen Ergebnisse/Ausgaben des Programms in einer Textdatei hinzu.
- Komplette Punktzahl kann nur bei den Aufgaben mit vollständigem Lösungsweg d.h. ausführlichen Kommentaren im Quellcode erzielt werden.
- Fragen zur Bearbeitung können Sie auf WueCampus, per Email oder persönlich an Tobias Herbst oder Dominik Regele stellen.